



APRESENTANDO T.O.D.A.N

A Nova Arena Assíncrona.

Reinventando o Clássico.

Movimente seu ciclo social criando dinâmicas personalizadas e apimentando a brincadeira.



Verdade.

Força uma resposta honesta.



Limites de Tempo



Desafio.

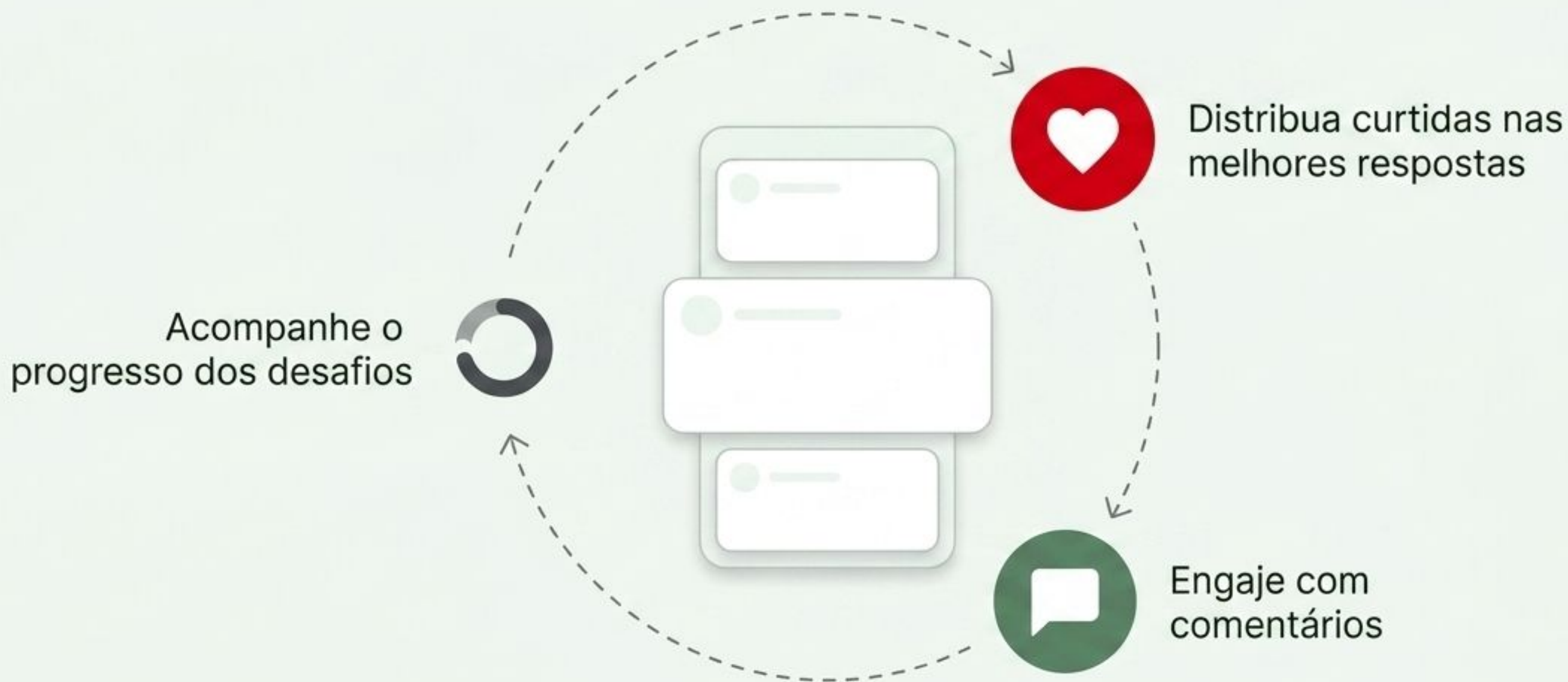
Desafia amigos a agir na vida real.



Quantidade de Tentativas

Muito Além de um Jogo Casual.

Todas as resoluções e criações alimentam um feed dinâmico e unificado. Uma verdadeira rede social de entretenimento.



Conexão Sem Exigir Sincronia.



Provocações Diretas

Teste os limites enviando desafios para um amigo.



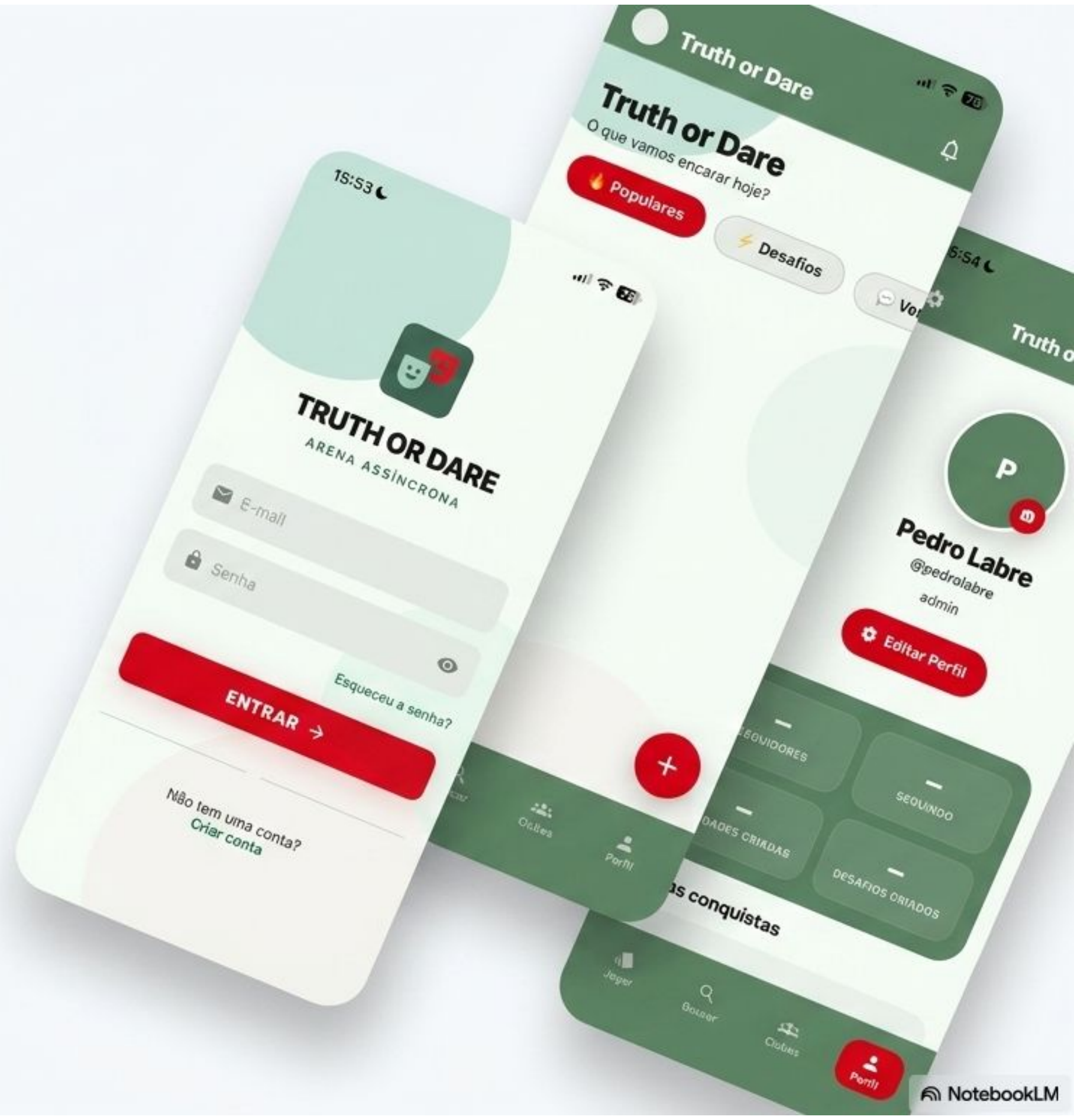
Clubes Exclusivos

Reúna a galera para competições em grupo e temáticas.

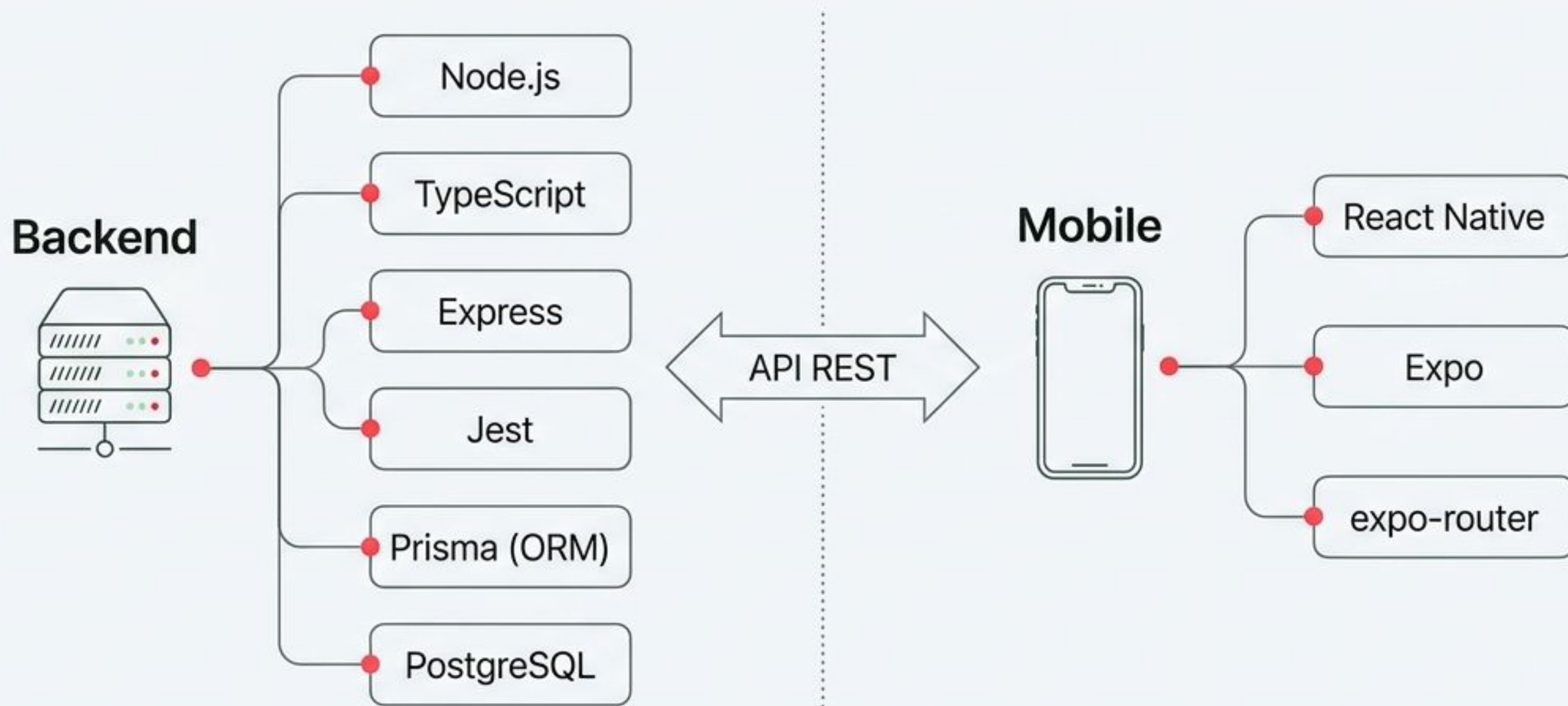
A diversão se mantém ativa gerando boas risadas no seu próprio ritmo, sem a necessidade de estarem todos online simultaneamente.

Truth or Dare: Guia Visual de Arquitetura

Mapeamento técnico e status de
integração do produto.



A Estrutura do Monorepo



O projeto é unificado, dividindo a lógica entre um backend robusto em Node.js e um frontend móvel escalável gerenciado via Expo Router.

O Coração dos Dados (Prisma)

A estrutura central atende a todas as frentes. O esquema relacional em PostgreSQL gerencia modelos como User, Truth, Dare e Club.



Challenges

```
model Dare {  
  id          String @id @default(uuid())  
  authorId    String  
  targetUserId String  
  // ...  
  author      User    @relation("DareAuthor", fields: [authorId],  
                                references: [id])  
}
```

 Users



Para colocar a aplicação no ar, o servidor Node.js está hospedado no Render utilizando os scripts já configurados no projeto, executando o processo de compilação e inicialização do Express.



Paralelamente, o Supabase atua como o provedor do banco de dados PostgreSQL, e a integração entre o backend hospedado no Render e o banco seria feita de forma transparente apenas **ajustando as variáveis de ambiente (como DATABASE_URL e DIRECT_URL)** do Prisma para as credenciais fornecidas pelo Supabase, garantindo um deploy robusto e escalável.

Autenticação Segura com JWT



15:53



CRIAR CONTA

ENTRE NA ARENA ASSÍNCRONA

☐ Eu concordo com os [Termos & Condições](#) e a [Política de Privacidade](#).

COMEÇAR A JOGAR ▶

Já tem uma conta?
[Entrar](#)



15:53



TRUTH OR DARE

ARENA ASSÍNCRONA

[Esqueceu a senha?](#)

ENTRAR →

Não tem uma conta?
[Criar conta](#)

Dados capturados e enviados via `fetch` para `/auth/login` e `/auth/signup`

Senhas criptografadas via `bcrypt` e tokens assinados com `jsonwebtoken` no serviço `auth.service.ts`

Token de acesso armazenado localmente usando `AsyncStorage`

Mecânica de Criação e Validação



O roteamento gerencia a escolha entre as entidades ``truth`` e ``dare``



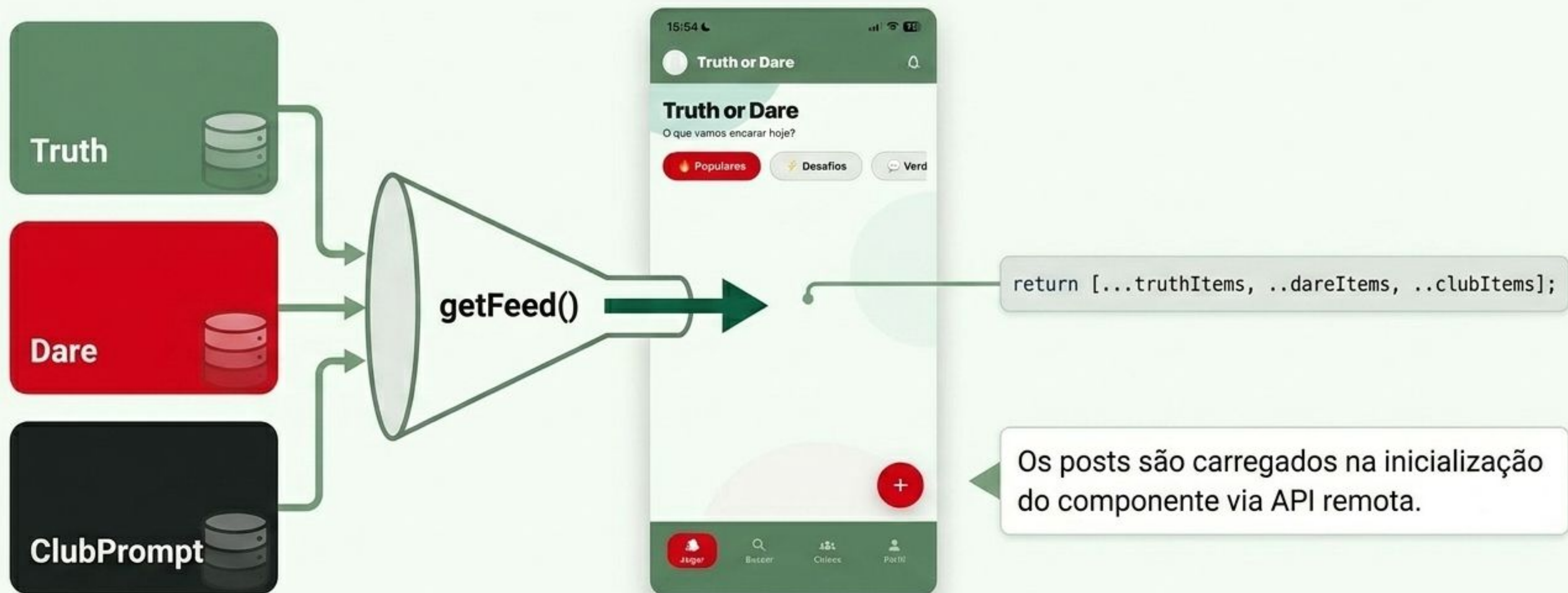
Parâmetros como texto, alvo, tentativas (``maxAttempts``) e tempo limite (``expiresAt``) são compilados



Ao clicar em 'Enviar', os dados atingem ``dares.routes.ts`` ou ``truths.routes.ts``, sendo validados e gravados via Prisma.

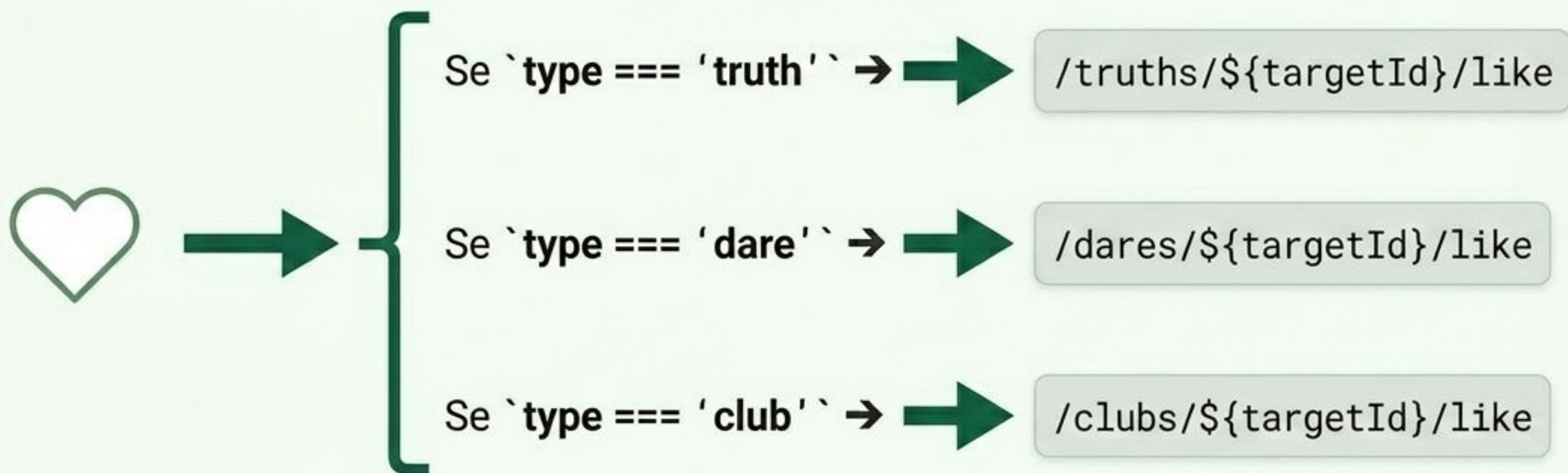
Visão Unificada no Feed

A página principal não lê apenas uma tabela. O serviço `feed.service.ts` agrupa os últimos itens gerados pelas 3 categorias principais.



Interação Polimórfica (Curtidas)

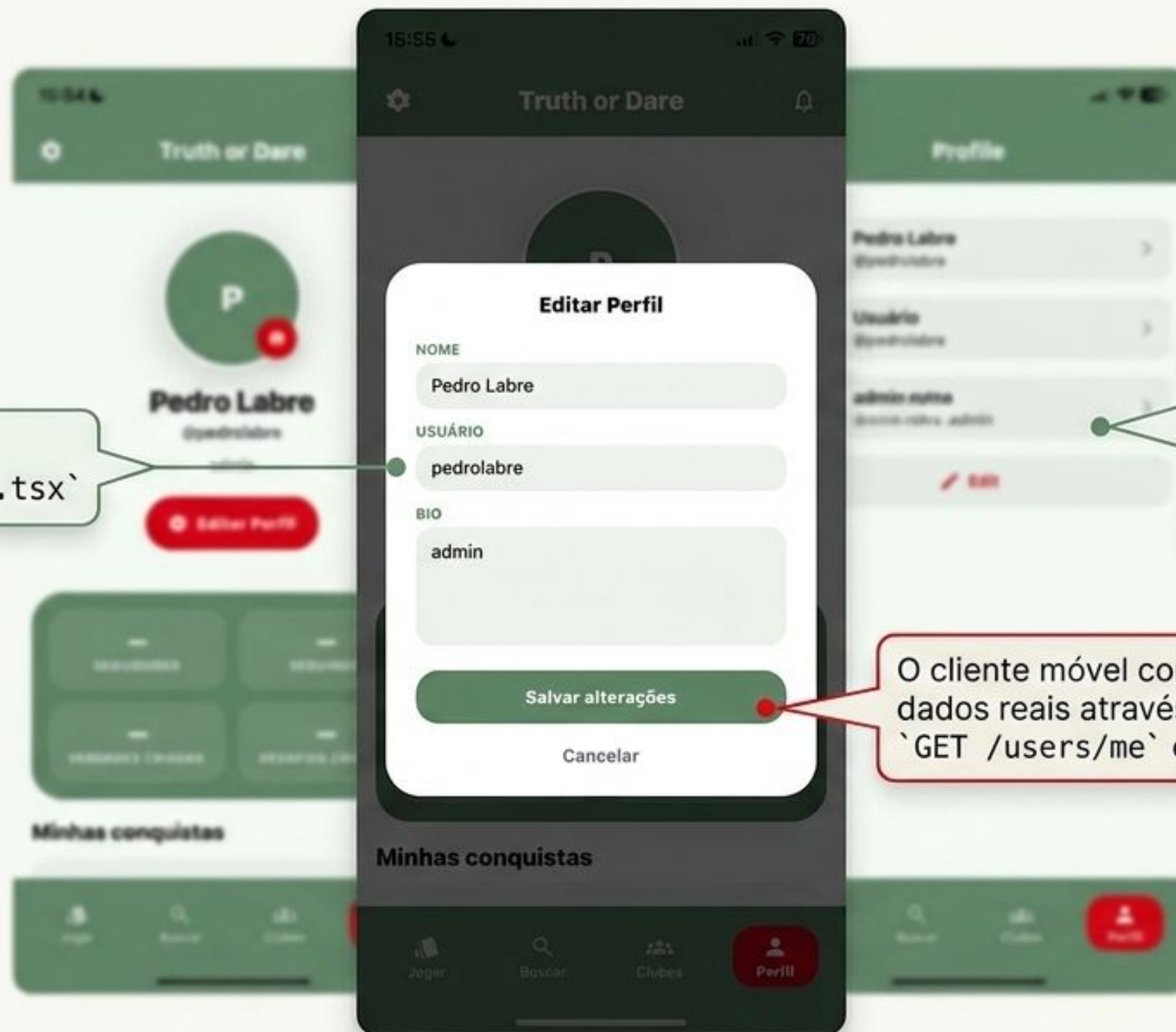
O sistema suporta interações em diferentes tipos de conteúdo através de um único modelo de dados. A tabela ``Like`` usa o enum ``LikeTargetType``.



O backend usa ``toggleLike`` para criar ou deletar o registro instantaneamente baseado em sua existência prévia.

Gestão de Identidade (Perfil)

Edição visual via
'ProfileEditModal.tsx'



A lógica de chamada da API é encapsulada de forma limpa no hook customizado `'useProfileScreen.ts'`.

O cliente móvel consome e atualiza dados reais através das rotas `'GET /users/me'` e `'PUT /users/me'`

Dinâmica de Cores (Theme Context)

O tema global evita estilos nativos complexos. Ele é gerenciado em tempo de execução validando o booleano `isDark`.

```
const { isDark } = useTheme();
const c = isDark ? DARK : LIGHT;
```

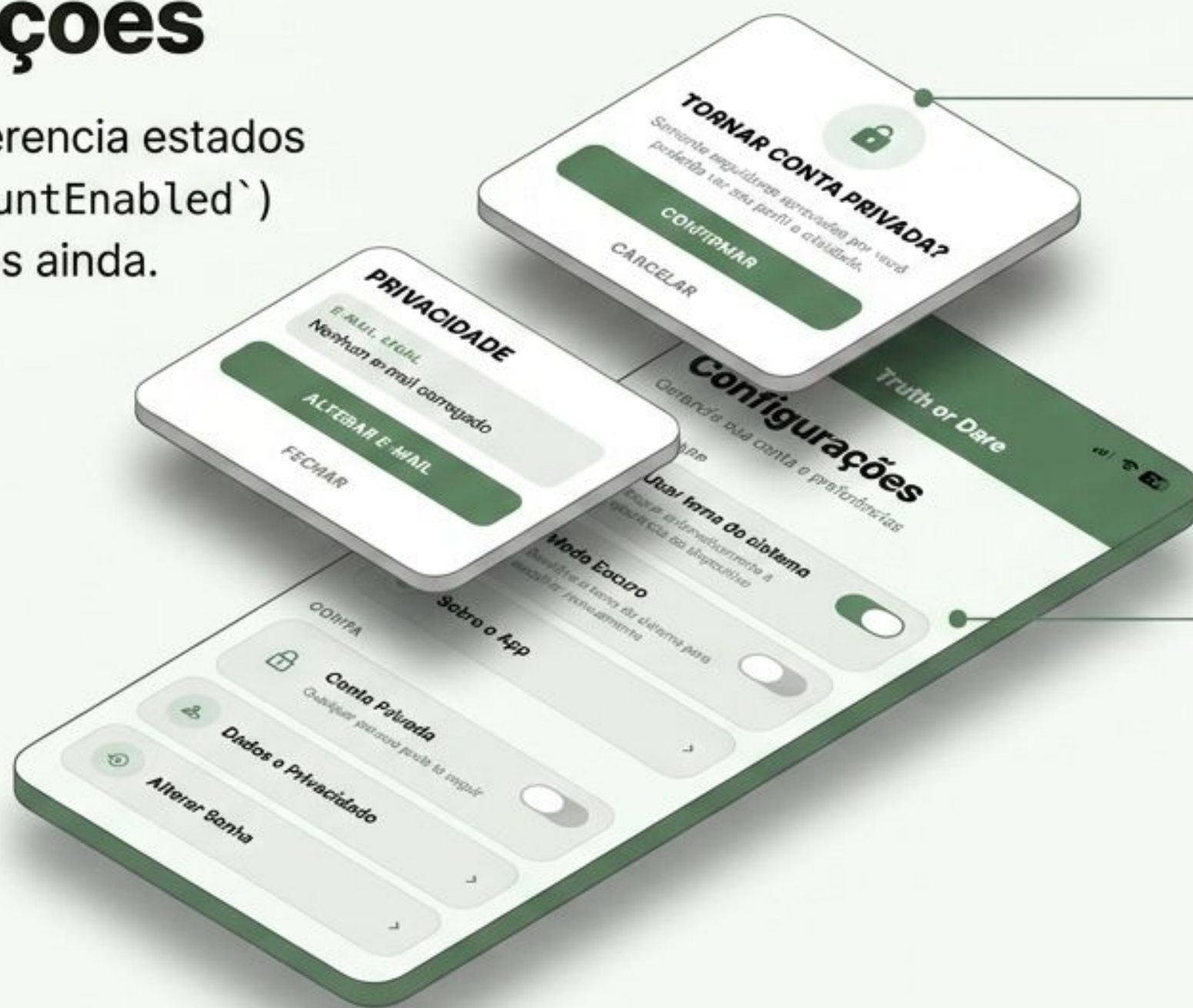


AS PALETAS SÃO DEFINIDAS EM CONSTANTES ISOLADAS (EX: `clubsTheme.ts`, `searchTheme.ts`), PERMITINDO MODOS 'SYSTEM', 'LIGHT' E 'DARK'.

⌚ APENAS FRONTEND

Arquitetura Modular de Configurações

A central de preferências gerencia estados locais (como `privateAccountEnabled`) sem mutar o banco de dados ainda.



A interface sobrepõe dezenas de modais independentes (ex: `SettingsPrivacyModal`).

Transição gerenciada por funções locais de estado: `openModal(modal)` e `switchModal(modal)`.

Clubes e Grupos: Prontos para Integração

O esquema Prisma já possui os modelos (`Club` , `ClubMember` , `ClubPrompt`) e suas migrações. Porém, a interface opera em modo isolado.



ESTADO DO HOOK

O comportamento atual é regido por valores fixos no hook `useClubsScreen.ts`:

`isLoading: false`

`isMyClubsEmpty: true`

Interface de Buscas Estruturada



```
{  
  users: [],  
  clubs: []  
}
```



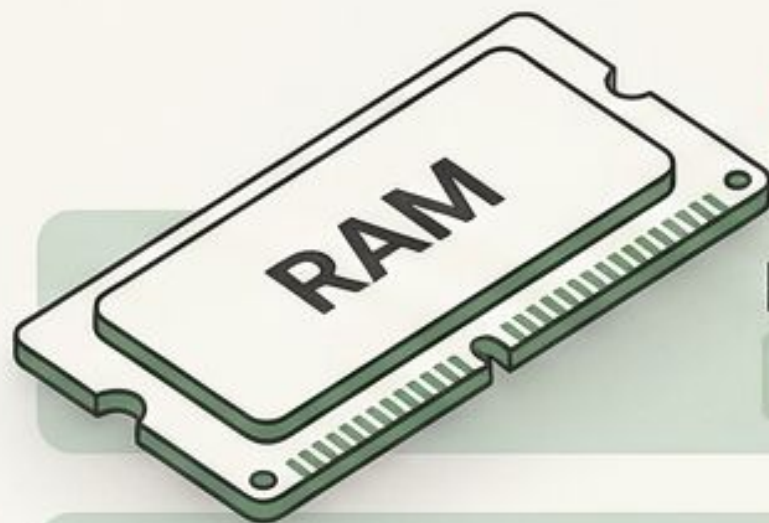
Visual totalmente estruturado e interativo em `search.tsx`

Atualmente inicializa com arrays vazios (`users: []`, `clubs: []`) geridos pelo hook `useSearchScreen.ts`.

Aguardando implementação do fetch na função `getUsers` do serviço da API.

Notificações em Memória

A área de atividades não efetua chamadas à API no momento.



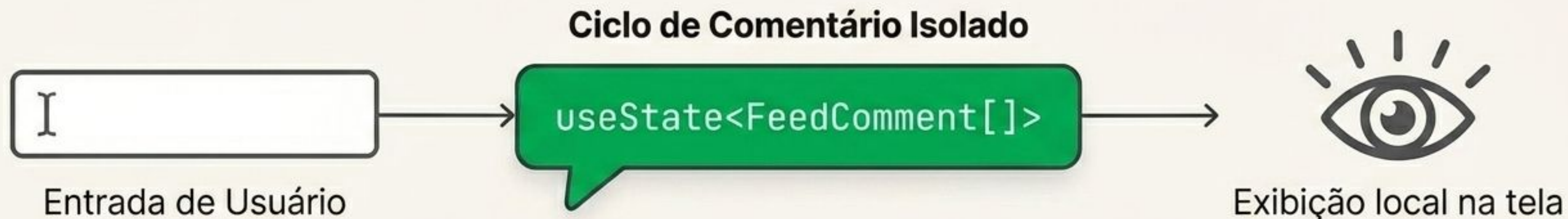
Exibe o componente estático
`NotificationsComingSoonCard`.

O arquivo inicializa um **array vazio** na memória
(`const notifications: unknown[] = []`).



Sistema de Comentários Temporário

O sistema que lida com comentários no feed opera 100% diretamente na memória do dispositivo (`feed-comments.tsx`).



Considerações Finais:

- Interações como `handleLikeComment` alteram apenas o estado visual.
- Não há conexão com persistência: o modelo relacional atual no diretório `/backend` não possui tabela equivalente a 'Comments'.

Matriz de Status e Integração

Funcionalidade	UI Pronta	Lógica Local	Banco de Dados (Prisma)	API Integrada
Autenticação	✓	✓	✓	✓
Mecânica de Desafios	✓	✓	✓	✓
Feed & Curtidas	✓	✓	✓	✓
Perfil de Usuário	✓	✓	✓	✓
Clubes & Grupos	✓	✓	✓	○
Busca global	✓	✓	○	○
Notificações	✓	✓	○	○
Comentários	✓	✓	○	○
Configurações/Modais	✓	✓	○	○

A base arquitetural sólida permite rápida conexão das áreas pendentes à API unificada.

Próximos Passos: O Caminho para a Coesão

